

$$A[j] \leq A[j + g] \leq A[j + 2g] \leq A[j + 3g] \leq A[j + 4g].$$

Each vertical column in Figure 9.3 depicts a sorted group of 5 elements. The median of each 5-element group is $A[j + 2g]$, and thus all the 5-element medians, shown in red, lie in the range $A[p + 2g : p + 3g - 1]$.

Next, line 16 determines the pivot x by recursively calling SELECT to find the median (specifically, the $\lceil g/2 \rceil$ th smallest) of the g group medians. Line 17 uses the modified PARTITION-AROUND algorithm to partition the elements of $A[p : r]$ around x , returning the index q of x , so that $A[q] = x$, elements in $A[p : q]$ are all at most x , and elements in $A[q : r]$ are greater than or equal to x .

The remainder of the code mirrors that of RANDOMIZED-SELECT. If the pivot x is the i th largest, the procedure returns it. Otherwise, the procedure recursively calls itself on either $A[p : q - 1]$ or $A[q + 1 : r]$, depending on the value of i .

Let's analyze the running time of SELECT and see how the judicious choice of the pivot x plays into a guarantee on its worst-case running time.

Theorem 9.3

The running time of SELECT on an input of n elements is $\Theta(n)$.

Proof Define $T(n)$ as the worst-case time to run SELECT on any input subarray $A[p : r]$ of size at most n , that is, for which $r - p + 1 \leq n$. By this definition, $T(n)$ is monotonically increasing.

We first determine an upper bound on the time spent outside the recursive calls in lines 16, 23, and 24. The **while** loop in lines 1–10 executes 0 to 4 times, which is $O(1)$ times. Since the dominant time within the loop is the computation of the minimum in lines 2–4, which takes $\Theta(n)$ time, lines 1–10 execute in $O(1) \cdot \Theta(n) = O(n)$ time. The sorting of the 5-element groups in lines 12–13 takes $\Theta(n)$ time because each 5-element group takes $\Theta(1)$ time to sort (even using an asymptotically inefficient sorting algorithm such as insertion sort), and there are g elements to sort, where $n/5 - 1 < g \leq n/5$. Finally, the time to partition in line 17 is $\Theta(n)$, as Exercise 7.1-3 on page 187 asks you to show. Because the remaining bookkeeping only costs $\Theta(1)$ time, the total amount of time spent outside of the recursive calls is $O(n) + \Theta(n) + \Theta(n) + \Theta(1) = \Theta(n)$.

Now let's determine the running time for the recursive calls. The recursive call to find the pivot in line 16 takes $T(g) \leq T(n/5)$ time, since $g \leq n/5$ and $T(n)$ monotonically increases. Of the two recursive calls in lines 23 and 24, at most one is executed. But we'll see that no matter which of these two recursive calls to SELECT actually executes, the number of elements in the recursive call turns out to be at most $7n/10$, and hence the worst-case cost for lines 23 and 24 is at most $T(7n/10)$. Let's now show that the machinations with group medians and the choice of the pivot x as the median of the group medians guarantees this property.

Figure 9.3 helps to visualize what's going on. There are $g \leq n/5$ groups of 5 elements, with each group shown as a column sorted from bottom to top. The arrows show the ordering of elements within the columns. The columns are ordered from left to right with groups to the left of x 's group having a group median less than x and those to the right of x 's group having a group median greater than x . Although the relative order within each group matters, the relative order among groups to the left of x 's column doesn't really matter, and neither does the relative order among groups to the right of x 's column. The important thing is that the groups to the left have group medians less than x (shown by the horizontal arrows entering x), and that the groups to the right have group medians greater than x (shown by the horizontal arrows leaving x). Thus, the yellow region contains elements that we know are greater than or equal to x , and the blue region contains elements that we know are less than or equal to x .

These two regions each contain at least $3g/2$ elements. The number of group medians in the yellow region is $\lfloor g/2 \rfloor + 1$, and for each group median, two additional elements are greater than it, making a total of $3(\lfloor g/2 \rfloor + 1) \geq 3g/2$ elements. Similarly, the number of group medians in the blue region is $\lceil g/2 \rceil$, and for each group median, two additional elements are less than it, making a total of $3\lceil g/2 \rceil \geq 3g/2$.

The elements in the yellow region cannot fall into the low side of the partition around x , and those in the blue region cannot fall into the high side. The elements in neither region—those lying on a white background—could fall into either side of the partition. But since the low side of the partition excludes the elements in the yellow region, and there are a total of $5g$ elements, we know that the low side of the partition can contain at most $5g - 3g/2 = 7g/2 \leq 7n/10$ elements. Likewise, the high side of the partition excludes the elements in the blue region, and a similar calculation shows that it also contains at most $7n/10$ elements.

All of which leads to the following recurrence for the worst-case running time of SELECT:

$$T(n) \leq T(n/5) + T(7n/10) + \Theta(n). \quad (9.1)$$

We can show that $T(n) = O(n)$ by substitution.² More specifically, we'll prove that $T(n) \leq cn$ for some suitably large constant $c > 0$ and all $n > 0$. Substituting this inductive hypothesis into the right-hand side of recurrence (9.1) and assuming that $n \geq 5$ yields

² We could also use the Akra-Bazzi method from Section 4.7, which involves calculus, to solve this recurrence. Indeed, a similar recurrence (4.24) on page 117 was used to illustrate that method.

$$\begin{aligned}
T(n) &\leq c(n/5) + c(7n/10) + \Theta(n) \\
&\leq 9cn/10 + \Theta(n) \\
&= cn - cn/10 + \Theta(n) \\
&\leq cn
\end{aligned}$$

if c is chosen large enough that $c/10$ dominates the upper-bound constant hidden by the $\Theta(n)$. In addition to this constraint, we can pick c large enough that $T(n) \leq cn$ for all $n \leq 4$, which is the base case of the recursion within SELECT. The running time of SELECT is therefore $O(n)$ in the worst case, and because line 13 alone takes $\Theta(n)$ time, the total time is $\Theta(n)$. ■

As in a comparison sort (see Section 8.1), SELECT and RANDOMIZED-SELECT determine information about the relative order of elements only by comparing elements. Recall from Chapter 8 that sorting requires $\Omega(n \lg n)$ time in the comparison model, even on average (see Problem 8-1). The linear-time sorting algorithms in Chapter 8 make assumptions about the type of the input. In contrast, the linear-time selection algorithms in this chapter do not require any assumptions about the input's type, only that the elements are distinct and can be pairwise compared according to a linear order. The algorithms in this chapter are not subject to the $\Omega(n \lg n)$ lower bound, because they manage to solve the selection problem without sorting all the elements. Thus, solving the selection problem by sorting and indexing, as presented in the introduction to this chapter, is asymptotically inefficient in the comparison model.

Exercises

9.3-1

In the algorithm SELECT, the input elements are divided into groups of 5. Show that the algorithm works in linear time if the input elements are divided into groups of 7 instead of 5.

9.3-2

Suppose that the preprocessing in lines 1–10 of SELECT is replaced by a base case for $n \geq n_0$, where n_0 is a suitable constant; that g is chosen as $\lfloor (r - p + 1)/5 \rfloor$; and that the elements in $A[5g : n]$ belong to no group. Show that although the recurrence for the running time becomes messier, it still solves to $\Theta(n)$.

9.3-3

Show how to use SELECT as a subroutine to make quicksort run in $O(n \lg n)$ time in the worst case, assuming that all elements are distinct.

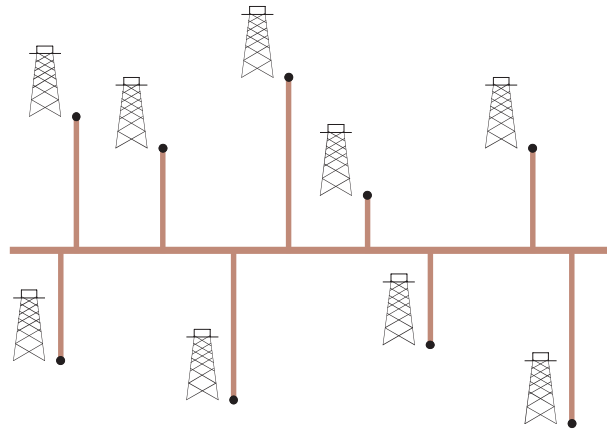


Figure 9.4 Professor Olay needs to determine the position of the east-west oil pipeline that minimizes the total length of the north-south spurs.

★ **9.3-4**

Suppose that an algorithm uses only comparisons to find the i th smallest element in a set of n elements. Show that it can also find the $i - 1$ smaller elements and the $n - i$ larger elements without performing any additional comparisons.

9.3-5

Show how to determine the median of a 5-element set using only 6 comparisons.

9.3-6

You have a “black-box” worst-case linear-time median subroutine. Give a simple, linear-time algorithm that solves the selection problem for an arbitrary order statistic.

9.3-7

Professor Olay is consulting for an oil company, which is planning a large pipeline running east to west through an oil field of n wells. The company wants to connect a spur pipeline from each well directly to the main pipeline along a shortest route (either north or south), as shown in Figure 9.4. Given the x - and y -coordinates of the wells, how should the professor pick an optimal location of the main pipeline to minimize the total length of the spurs? Show how to determine an optimal location in linear time.

9.3-8

The k th *quantiles* of an n -element set are the $k - 1$ order statistics that divide the sorted set into k equal-sized sets (to within 1). Give an $O(n \lg k)$ -time algorithm to list the k th quantiles of a set.

9.3-9

Describe an $O(n)$ -time algorithm that, given a set S of n distinct numbers and a positive integer $k \leq n$, determines the k numbers in S that are closest to the median of S .

9.3-10

Let $X[1:n]$ and $Y[1:n]$ be two arrays, each containing n numbers already in sorted order. Give an $O(\lg n)$ -time algorithm to find the median of all $2n$ elements in arrays X and Y . Assume that all $2n$ numbers are distinct.

Problems**9-1 Largest i numbers in sorted order**

You are given a set of n numbers, and you wish to find the i largest in sorted order using a comparison-based algorithm. Describe the algorithm that implements each of the following methods with the best asymptotic worst-case running time, and analyze the running times of the algorithms in terms of n and i .

- a. Sort the numbers, and list the i largest.
- b. Build a max-priority queue from the numbers, and call EXTRACT-MAX i times.
- c. Use an order-statistic algorithm to find the i th largest number, partition around that number, and sort the i largest numbers.

9-2 Variant of randomized selection

Professor Mendel has proposed simplifying RANDOMIZED-SELECT by eliminating the check for whether i and k are equal. The simplified procedure is SIMPLER-RANDOMIZED-SELECT.

```

SIMPLER-RANDOMIZED-SELECT( $A, p, r, i$ )
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i \leq k$ 
6      return SIMPLER-RANDOMIZED-SELECT( $A, p, q, i$ )
7  else return SIMPLER-RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )

```

- a. Argue that in the worst case, SIMPLER-RANDOMIZED-SELECT never terminates.
- b. Prove that the expected running time of SIMPLER-RANDOMIZED-SELECT is still $O(n)$.

9-3 Weighted median

Consider n elements $x_1, x_2, \dots, x_n$ with positive weights $w_1, w_2, \dots, w_n$ such that $\sum_{i=1}^n w_i = 1$. The **weighted (lower) median** is an element x_k satisfying

$$\sum_{x_i < x_k} w_i < \frac{1}{2}$$

and

$$\sum_{x_i > x_k} w_i \leq \frac{1}{2}.$$

For example, consider the following elements x_i and weights w_i :

i	1	2	3	4	5	6	7
x_i	3	8	2	5	4	1	6
w_i	0.12	0.35	0.025	0.08	0.15	0.075	0.2

For these elements, the median is $x_5 = 4$, but the weighted median is $x_7 = 6$. To see why the weighted median is x_7 , observe that the elements less than x_7 are x_1, x_3, x_4, x_5 , and x_6 , and the sum $w_1 + w_3 + w_4 + w_5 + w_6 = 0.45$, which is less than $1/2$. Furthermore, only element x_2 is greater than x_7 , and $w_2 = 0.35$, which is no greater than $1/2$.

- a. Argue that the median of $x_1, x_2, \dots, x_n$ is the weighted median of the x_i with weights $w_i = 1/n$ for $i = 1, 2, \dots, n$.
- b. Show how to compute the weighted median of n elements in $O(n \lg n)$ worst-case time using sorting.
- c. Show how to compute the weighted median in $\Theta(n)$ worst-case time using a linear-time median algorithm such as SELECT from Section 9.3.

The **post-office location problem** is defined as follows. The input is n points $p_1, p_2, \dots, p_n$ with associated weights $w_1, w_2, \dots, w_n$. A solution is a point p (not necessarily one of the input points) that minimizes the sum $\sum_{i=1}^n w_i d(p, p_i)$, where $d(a, b)$ is the distance between points a and b .

- d. Argue that the weighted median is a best solution for the one-dimensional post-office location problem, in which points are simply real numbers and the distance between points a and b is $d(a, b) = |a - b|$.
- e. Find the best solution for the two-dimensional post-office location problem, in which the points are (x, y) coordinate pairs and the distance between points $a = (x_1, y_1)$ and $b = (x_2, y_2)$ is the **Manhattan distance** given by $d(a, b) = |x_1 - x_2| + |y_1 - y_2|$.

9-4 Small order statistics

Let's denote by $S(n)$ the worst-case number of comparisons used by SELECT to select the i th order statistic from n numbers. Although $S(n) = \Theta(n)$, the constant hidden by the Θ -notation is rather large. When i is small relative to n , there is an algorithm that uses SELECT as a subroutine but makes fewer comparisons in the worst case.

- a. Describe an algorithm that uses $U_i(n)$ comparisons to find the i th smallest of n elements, where

$$U_i(n) = \begin{cases} S(n) & \text{if } i \geq n/2, \\ \lfloor n/2 \rfloor + U_i(\lceil n/2 \rceil) + S(2i) & \text{otherwise.} \end{cases}$$

(Hint: Begin with $\lfloor n/2 \rfloor$ disjoint pairwise comparisons, and recurse on the set containing the smaller element from each pair.)

- b. Show that, if $i < n/2$, then $U_i(n) = n + O(S(2i) \lg(n/i))$.
- c. Show that if i is a constant less than $n/2$, then $U_i(n) = n + O(\lg n)$.
- d. Show that if $i = n/k$ for $k \geq 2$, then $U_i(n) = n + O(S(2n/k) \lg k)$.

9-5 Alternative analysis of randomized selection

In this problem, you will use indicator random variables to analyze the procedure RANDOMIZED-SELECT in a manner akin to our analysis of RANDOMIZED-QUICKSORT in Section 7.4.2.

As in the quicksort analysis, we assume that all elements are distinct, and we rename the elements of the input array A as $z_1, z_2, \dots, z_n$, where z_i is the i th smallest element. Thus the call $\text{RANDOMIZED-SELECT}(A, 1, n, i)$ returns z_i .

For $1 \leq j < k \leq n$, let

$$X_{ijk} = \mathbf{I}\{z_j \text{ is compared with } z_k \text{ sometime during the execution of the algorithm to find } z_i\}.$$

- a. Give an exact expression for $E[X_{ijk}]$. (*Hint:* Your expression may have different values, depending on the values of i , j , and k .)
- b. Let X_i denote the total number of comparisons between elements of array A when finding z_i . Show that

$$E[X_i] \leq 2 \left(\sum_{j=1}^i \sum_{k=i}^n \frac{1}{k-j+1} + \sum_{k=i+1}^n \frac{k-i-1}{k-i+1} + \sum_{j=1}^{i-2} \frac{i-j-1}{i-j+1} \right).$$
- c. Show that $E[X_i] \leq 4n$.
- d. Conclude that, assuming all elements of array A are distinct, RANDOMIZED-SELECT runs in $O(n)$ expected time.

9-6 Select with groups of 3

Exercise 9.3-1 asks you to show that the SELECT algorithm still runs in linear time if the elements are divided into groups of 7. This problem asks about dividing into groups of 3.

- a. Show that SELECT runs in linear time if you divide the elements into groups whose size is any odd constant greater than 3.
- b. Show that SELECT runs in $O(n \lg n)$ time if you divide the elements into groups of size 3.

Because the bound in part (b) is just an upper bound, we do not know whether the groups-of-3 strategy actually runs in $O(n)$ time. But by repeating the groups-of-3 idea on the middle group of medians, we can pick a pivot that guarantees $O(n)$ time. The SELECT3 algorithm on the next page determines the i th smallest of an input array of $n > 1$ distinct elements.

- c. Describe in English how the SELECT3 algorithm works. Include in your description one or more suitable diagrams.
- d. Show that SELECT3 runs in $O(n)$ time in the worst case.

Chapter notes

The worst-case linear-time median-finding algorithm was devised by Blum, Floyd, Pratt, Rivest, and Tarjan [62]. The fast randomized version is due to Hoare [218]. Floyd and Rivest [147] have developed an improved randomized version that partitions around an element recursively selected from a small sample of the elements.


```

SELECT3( $A, p, r, i$ )
1  while  $(r - p + 1) \bmod 9 \neq 0$ 
2      for  $j = p + 1$  to  $r$                                 // put the minimum into  $A[p]$ 
3          if  $A[p] > A[j]$ 
4              exchange  $A[p]$  with  $A[j]$ 
5      // If we want the minimum of  $A[p : r]$ , we're done.
6      if  $i == 1$ 
7          return  $A[p]$ 
8      // Otherwise, we want the  $(i - 1)$ st element of  $A[p + 1 : r]$ .
9       $p = p + 1$ 
10      $i = i - 1$ 
11      $g = (r - p + 1)/3$                                 // number of 3-element groups
12     for  $j = p$  to  $p + g - 1$                             // run through the groups
13         sort  $\langle A[j], A[j + g], A[j + 2g] \rangle$  in place
14     // All group medians now lie in the middle third of  $A[p : r]$ .
15      $g' = g/3$                                           // number of 3-element subgroups
16     for  $j = p + g$  to  $p + g + g' - 1$                 // sort the subgroups
17         sort  $\langle A[j], A[j + g'], A[j + 2g'] \rangle$  in place
18     // All subgroup medians now lie in the middle ninth of  $A[p : r]$ .
19     // Find the pivot  $x$  recursively as the median of the subgroup medians.
20      $x = \text{SELECT3}(A, p + 4g', p + 5g' - 1, \lceil g'/2 \rceil)$ 
21      $q = \text{PARTITION-AROUND}(A, p, r, x)$  // partition around the pivot
22     // The rest is just like lines 19–24 of SELECT.
23      $k = q - p + 1$ 
24     if  $i == k$ 
25         return  $A[q]$                                 // the pivot value is the answer
26     elseif  $i < k$ 
27         return  $\text{SELECT3}(A, p, q - 1, i)$ 
28     else return  $\text{SELECT3}(A, q + 1, r, i - k)$ 

```

It is still unknown exactly how many comparisons are needed to determine the median. Bent and John [48] gave a lower bound of $2n$ comparisons for median finding, and Schönhage, Paterson, and Pippenger [397] gave an upper bound of $3n$. Dor and Zwick have improved on both of these bounds. Their upper bound [123] is slightly less than $2.95n$, and their lower bound [124] is $(2 + \epsilon)n$, for a small positive constant ϵ , thereby improving slightly on related work by Dor et al. [122]. Paterson [354] describes some of these results along with other related work.

Problem 9-6 was inspired by a paper by Chen and Dumitrescu [84].



Part III Data Structures